```
    void call_once(once_flag& flag,Callable func,Args args...);
}
```

D.5.1. std::mutex class

The `std::mutex` class provides a basic mutual exclusion and synchronization facility for threads that can be used to protect shared data. Prior to accessing the data protected by the mutex, the mutex must be *locked* by calling `lock()` or `try_lock()`. Only one thread may hold the lock at a time, so if another thread also tries to lock the mutex, it will fail (`try_lock()`) or block (`lock()`) as appropriate. Once a thread is done accessing the shared data, it then must call `unlock()` to release the lock and allow other threads to acquire it.

`std::mutex` meets the `Lockable` requirements.

Class definition

```
class mutex
{
public:
    mutex(mutex const&)=delete;
    mutex& operator=(mutex const&)=delete;

    constexpr mutex() noexcept;
    ~mutex();

    void lock();
    void unlock();
    bool try_lock();
};
```

Std::Mutex Default Constructor

Constructs a `std::mutex` object.

Declaration

```
constexpr mutex() noexcept;
```

Effects